CSC485 A2

Shilin Zhang

November 2024

1 Question 0

1.1 a

```
Deepest Synset Synset('rock_hind.n.01') with Max Depth 19
Path 1:
  Synset('entity.n.01') (Depth: 0)
  Synset('physical_entity.n.01') (Depth: 1)
  Synset('object.n.01') (Depth: 2)
  Synset('whole.n.02') (Depth: 3)
  Synset('living_thing.n.01') (Depth: 4)
  Synset('organism.n.01') (Depth: 5)
  Synset('animal.n.01') (Depth: 6)
  Synset('chordate.n.01') (Depth: 7)
  Synset('vertebrate.n.01') (Depth: 8)
  Synset('aquatic_vertebrate.n.01') (Depth: 9)
  Synset('fish.n.01') (Depth: 10)
  Synset('food_fish.n.01') (Depth: 11)
  Synset('sea_bass.n.02') (Depth: 12)
  Synset('grouper.n.02') (Depth: 13)
  Synset('hind.n.01') (Depth: 14)
  Synset('rock_hind.n.01') (Depth: 15)
Path 2:
  Synset('entity.n.01') (Depth: 0)
  Synset('physical_entity.n.01') (Depth: 1)
  Synset('object.n.01') (Depth: 2)
  Synset('whole.n.02') (Depth: 3)
  Synset('living_thing.n.01') (Depth: 4)
  Synset('organism.n.01') (Depth: 5)
  Synset('animal.n.01') (Depth: 6)
  Synset('chordate.n.01') (Depth: 7)
  Synset('vertebrate.n.01') (Depth: 8)
  Synset('aquatic_vertebrate.n.01') (Depth: 9)
  Synset('fish.n.01') (Depth: 10)
  Synset('bony_fish.n.01') (Depth: 11)
  Synset('teleost_fish.n.01') (Depth: 12)
  Synset('spiny-finned_fish.n.01') (Depth: 13)
  Synset('percoid_fish.n.01') (Depth: 14)
  Synset('serranid_fish.n.01') (Depth: 15)
  Synset('sea_bass.n.02') (Depth: 16)
  Synset('grouper.n.02') (Depth: 17)
  Synset('hind.n.01') (Depth: 18)
  Synset('rock_hind.n.01') (Depth: 19)
```

2 Question 1

2.1 d

The extension is helpful because with this extension, we are no longer considering only the definition and examples of the word itself. We incorporated the definitions and examples of words similar to the word which gives us a larger and more comprehensive signature and background to work with.
By including these definitions and examples, we get a larger signature meaning that the likelihood of finding matching words in the context increases so the chance that the correct sense having a higher score than incorrect sense increases. So, the extended version of Lesk increases the potential for overlap with the context which leads to higher accuracy.

2.2 g

After implementing the two functions, the accuracy output indicate that the lesk_cos_oneside performs better than lesk_cos.
The difference between the two functions is that in lesk_cos_oneside, we construct the context and signature vectors by considering words that are in both the context and signature or only in context and not in signature whereas in lesk_cos, we also consider words that only in signature but not in context.
I believe this difference result in the improvement in accuracy because it reduces noise and irrelevant senses. In signature, it will sometimes include senses that are irrelevant to the context and lesk_cos_oneside removes these irrelevant senses by not considering words that are only in signature when constructing the vector which reduces the dimension of vector space to those words that are relevant to the context which will make a more accurate and reliable prediction compare to lesk_cos.
For example, if we are given a sentence like "He sat on the bank of the river" and we want to determine the meaning of the word bank, our signature will include the definition of the word bank that refers to financial institution as well as the definition of the word bank that refers to river bank.
If we use lesk_cos, then the denominator will be big because we include the irrelevant definition of financial institution in our signature vectors making the norm of it larger.
On the other hand, lesk_cos_oneside will have a smaller denominator because we do not consider the definition of financial institution in our signature vectors.
The numerator will be the same among the two functions since only words that appear in both context and signature contribute to the overall dot product so by not considering words that is in signature but not in context will not change the dot product which is the numerator.
This will cause the score of lesk_cos to be smaller than lesk_cos_oneside which means it is more likely that some incorrect sense will get a higher score than the correct sense if we use lesk_cos compared to lesk_cos_oneside.

So overall, the performance for lesk_cos_oneside is generally better than lesk_cos because it eliminates irrelevant senses (words in signature but not in context) when calculating the cosine similarity which makes the correct sense to have a higher similarity score and thus, have a higher probability of being selected.

2.3 h

If we design our vector in this way, the numerator of the CosSim is the dot product of the two vectors which is really just count how many spots where both vector has 1. It is really the same as set intersection of the two vectors. For the denominator, it is the product of the norm of the two vectors. The CosSim value will still be greatest when both sets have exactly the same elements and starts to decrease as elements in the two sets start to be different and eventually will be 0 if no element in the two sets are the same.

2.4 j

Normal Q1 Accuracy:

```
wolf:~/CSC485_A2$ python3 q1.py
/local/packages/python-3.12/lib/python3.12/site-packages/torch/cuda/__init__.py:619: UserWarning: Can't initialize NVML
  warnings.warn("Can't initialize NVML")
mfs: 41.6%
lesk: 39.3%
lesk_ext: 45.9%
lesk_cos: 38.3%
lesk_cos_onesided: 44.0%
lesk_w2v: 47.9%
```

Lower Q1 Accuracy:

```
wolf:~/CSC485_A2$ python3 q1lower.py
/local/packages/python-3.12/lib/python3.12/site-packages/torch/cuda/__init__.py:619: UserWarning: Can't initialize NVML
  warnings.warn("Can't initialize NVML")
mfs: 41.6%
lesk: 37.9%
lesk_ext: 46.0%
lesk_cos: 36.6%
lesk_cos_onesided: 44.0%
lesk_w2v: 46.9%
```

mfs

We can see that mfs does not change in performance because it just simply extract the most frequent senses of the word without looking at the meaning of the word and there is no word comparison involved so it is independent of token casing.

lesk

There is a slight drop for lesk function which I believe is caused by the loss of distinction between proper noun and common noun.

For example, Apple with first letter capitalized may be referring to the company but if we change it to lower case, it may lose the meaning of the company and only refer to the fruit apple.

lesk_ext

We see that lesk_ext increased by 0.1% where it does not experience the issue from lesk because it has a broad enough signature to be not effected as much

as lesk.
The broad signature allows lesk_ext to find the correct senses when we lose some information due to lower casing the context and signature.
Additionally, the accuracy even increased slightly.
I believe it is because in some cases, the first word in definition or example will be capitalized which may match the word in the context but since the word in context is not the first word, it may be lower case in context resulting in not matching.
So by lower casing all words, we eliminates this issue and I believe we did not see an improvement of this issue in accuracy of lesk because it only considers the definition and examples of the sense so this issue may not happen too often so there is no significant improvements observed.
But in context of lesk_ext, we have a lot more definition and examples where this issue could happen much more often so by lower casing all words, we are more likely to see an improvement of this issue and thus, leading to an increase in accuracy.

lesk_cos

We can observe a slight decrease in accuracy for lesk_cos algorithm and I think it is because of the same reason as lesk algorithm where by converting all words to lower case, we are losing distinction between proper noun and common noun.
In lesk_cos, we are using the broad signature as well but we did not observe an increase like lesk_ext and I believe it is because in lesk_ext, we are only looking at overlaps and with the reasoning for lesk_ext, overlap score will increase as we convert words to lower case.
However, in cosine similarity score, there is more factors that could change the score such as the normalization term or denominator.
If we lower case all words, we might get more overlap which increase the numerator but we could also increase the norm of both vector since we get more occurrence of words within context or signature.
This will make the cosine similarity score of the correct sense to increase less or even decrease if the norm increased more than the increase in numerator and due to the loss of distinction between proper and common words, the cosine similarity score for incorrect senses is more likely to be higher than the correct sense which leads to a slight decrease in accuracy.

lesk_cos_oneside

Unlike lesk_cos algorithm, we observed that the cos onside algorithm did not change in accuracy and I believe it is because the cos oneside algorithm focus on only words in context so both the impact of loss of distinction between proper and common words as well as the impact of increase in the two norms are minimized.
Because we are only considering words in context, the impact of lower casing will be minimized even though there is still probability that seeing a proper word in the context, but it is more unlikely comparing with seeing proper word in context or signature so the impact can be minimized in this way and due to the broad signature that we work with, we could still have enough information to get the correct sense similarly to analysis of lesk_ext.

Also, the issue of increase in norm will also be minimized since we are only working with words in context and our numerator will still increase by the same amount so our overall score for the correct sense is more likely to increase compared to lesk_cos.
Overall, the impact of lower casing all words is sufficiently minimized so we observed no change in accuracy.
lesk_w2v
We observed a slight decrease in accuracy and I believe it is simply because of the fact that we are losing information on proper words when we lower case all the words and we can see that the decrease is the minimum among all algorithm that experience a decrease so it still performs reasonably well even without case information.

3 Question 2

3.1 a

Yes. Context is really necessary when disambiguating word meanings. It is especially important when the word is ambiguous and the sense of the word depends heavily on the surrounding words or the context where the word is used. Those methods that we implement in Q1 rely on how often words appear, overlap and cosine similarity scores which does not consider order or syntactic dependencies so it is hard for them to solve ambiguities.
For example, the correct sense of a polysemous word often depends on the context in which it appears so by only considering wordforms and lemmata and not considering context or semantic roles, it will lead to incorrect disambiguation.
As a concrete example, we are given this sentence **"I could hear the ring of the bell from the school when I found my lost ring in my locker."**
In this example, the first ring refers to the sound made by the bell and the second ring refers to jewelry.
Without considering the context, methods that we implemented in Q1 will struggle between the two meanings of the word ring because they treat all words in the sentence equally without looking at the relationship between surrounding words and the target words and ignoring word order and syntactic relationships.
In this case, it is impossible that the above method will completely disambiguate the word ring since the only way to disambiguate them is to look at the context and surrounding words realizing that the first ring has the verb "hear" before it and the word "bell" after it so it refers to the meaning of the sound made by the bell whereas the second ring has the word "lost" describing it and the phrase "in my locker" stating the location of the ring so it refers to the jewelry.
As a general pattern, word order–invariant methods are not able to disambiguate homonyms or polysemous words where the meaning of the word can only be found by examining the syntactic and word relations between the target word and surrounding.

Thus, a static embeddings like word2vec is not effective in disambiguating homonyms or polysemous words because it always return the same vector for the same word regardless of the context that the word is used in whereas methods like BERT considers contextual information and can produce different vectors based on different context which helps to disambiguate homonyms or polysemous words.

3.2 c

We sort the corpus because we can effectively minimize the amount of padding that we add to each sentence.
While we train a model, we would like our input to have the same dimensions where in our case, we would like each sentence to have the same length. We can guarantee that all sentences have the same length by adding padding to sentences.
If we do not sort the corpus before batching, then there might be a significant differences in terms of the length of sentences in each batch. In this case, we will need to add a significant amount of padding to short sentences since we need them to have same length as long sentences in the batch. This will increase our input size which both takes up more memory and adds additional computational complexity to the model since these padding tokens does not mean anything and does not contribute to our predictions but our model still need to process it. So if we batch without sorting first, the model will be slow due to the additional computational complexity introduced by the significant amount of padding tokens that is added to short sentences.
On the other hand, if we sort the corpus before batching, we can effectively minimize the amount of padding added to each sentence in each batch because by sorting the corpus, we effectively minimizes the difference in length among sentences in each batch (minimizes the difference between the longest and shortest sentence in each batch). With this approach, we successfully minimizes the amount of padding added to each sentence and thus, minimizes the amount of complexity introduced by these padding tokens resulting in a faster runtime than leaving it as-is.

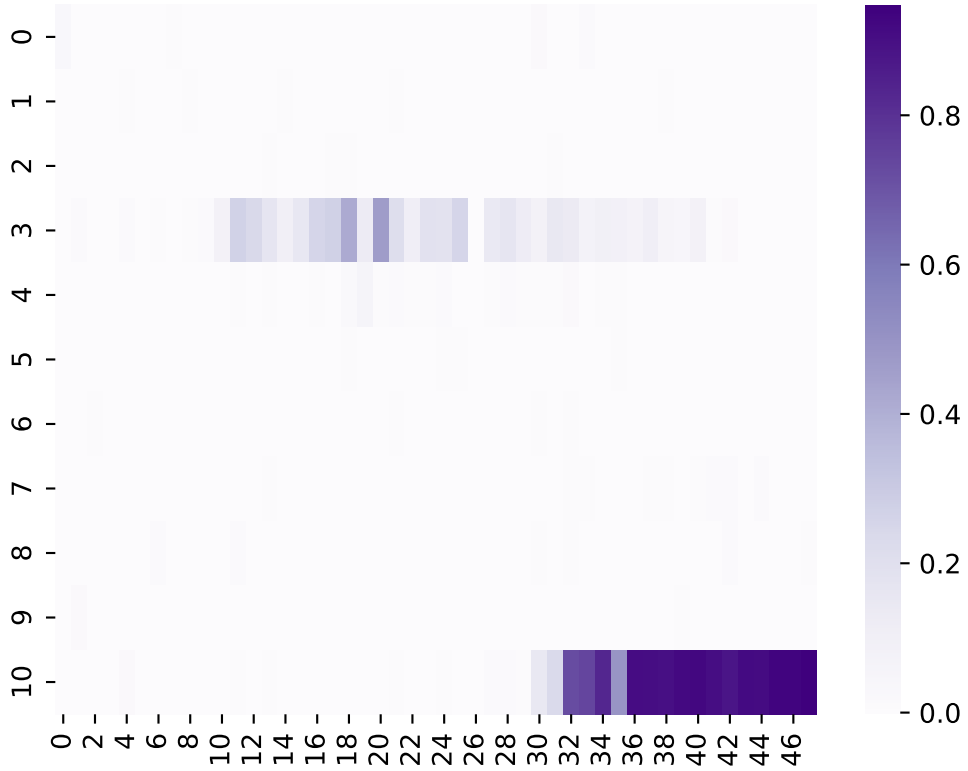
3.3 e

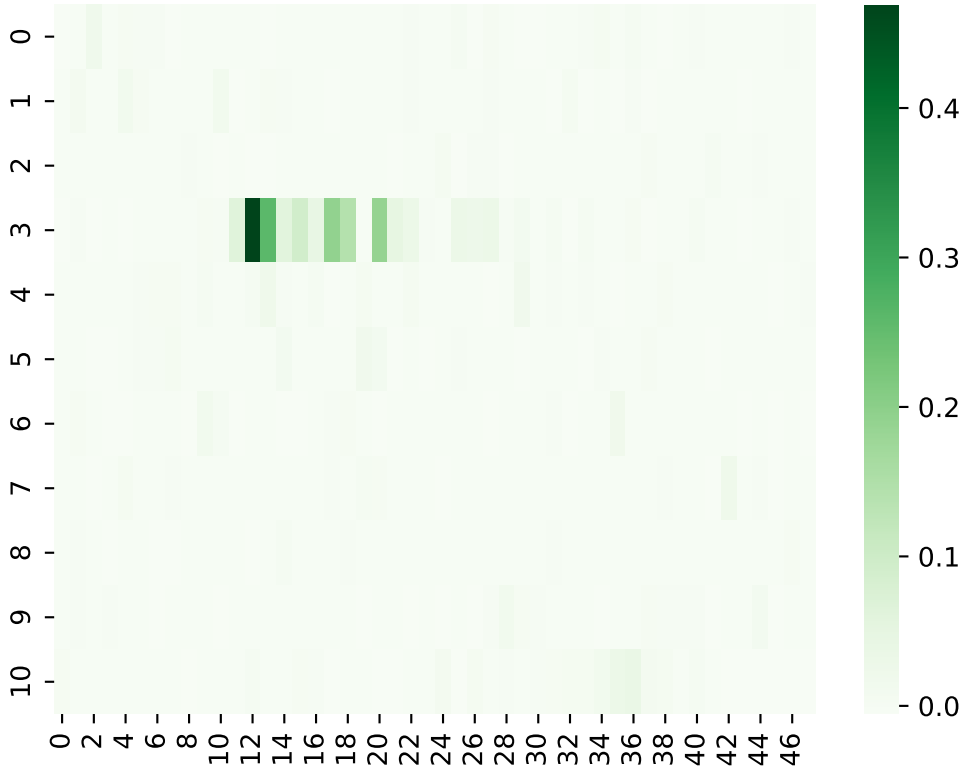
One issue with the lesk variants in Q1 is the fact that our algorithm depends on a database to disambiguate arbitrary sentences. Our algorithm from Q1 disambiguate sentences using synsets which is our database but we cannot guarantee that our database has all the word that exist so if the word does not exist in the database, then it is impossible to even make a decision on it's meaning. In real world, many words have disambiguate or may be misspelled intentionally as in memes so this issue could happen really often. In addition, the sentence may be misspelled by accident or the word may have other meaning that is not recorded

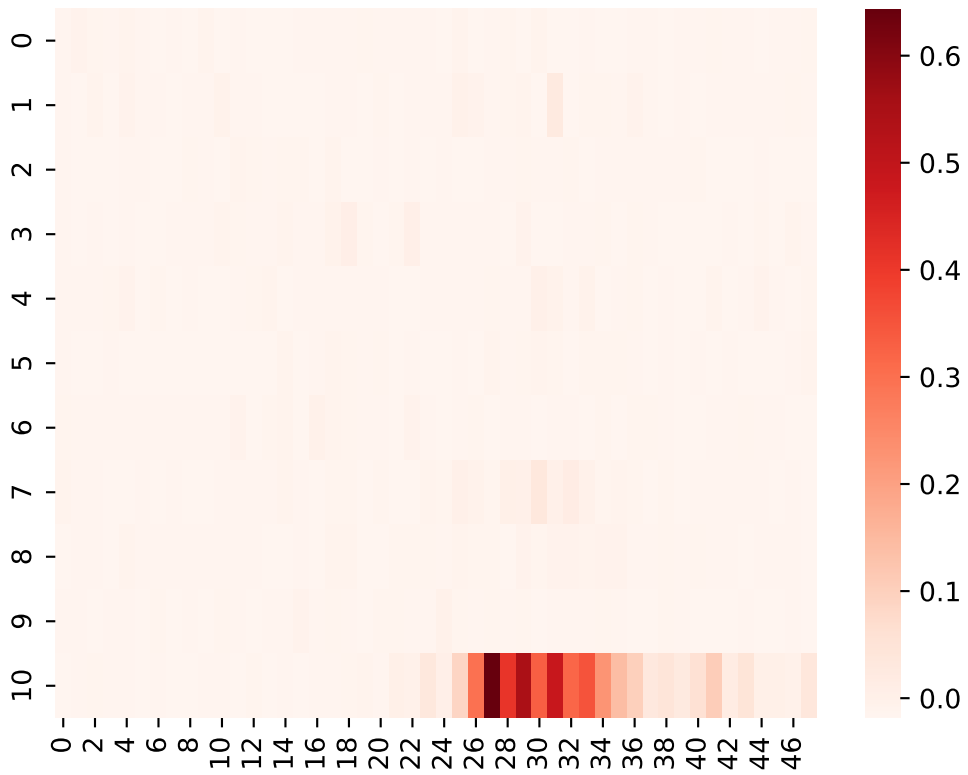
in the database. So this is an issue when disambiguate arbitrary sentences with database where we cannot guarantee the database includes every single word and every single meaning of each word and even the database can guarantee this, a misspell will make the algorithm not able to disambiguate the sentence.

4 Question 3

4.1 c

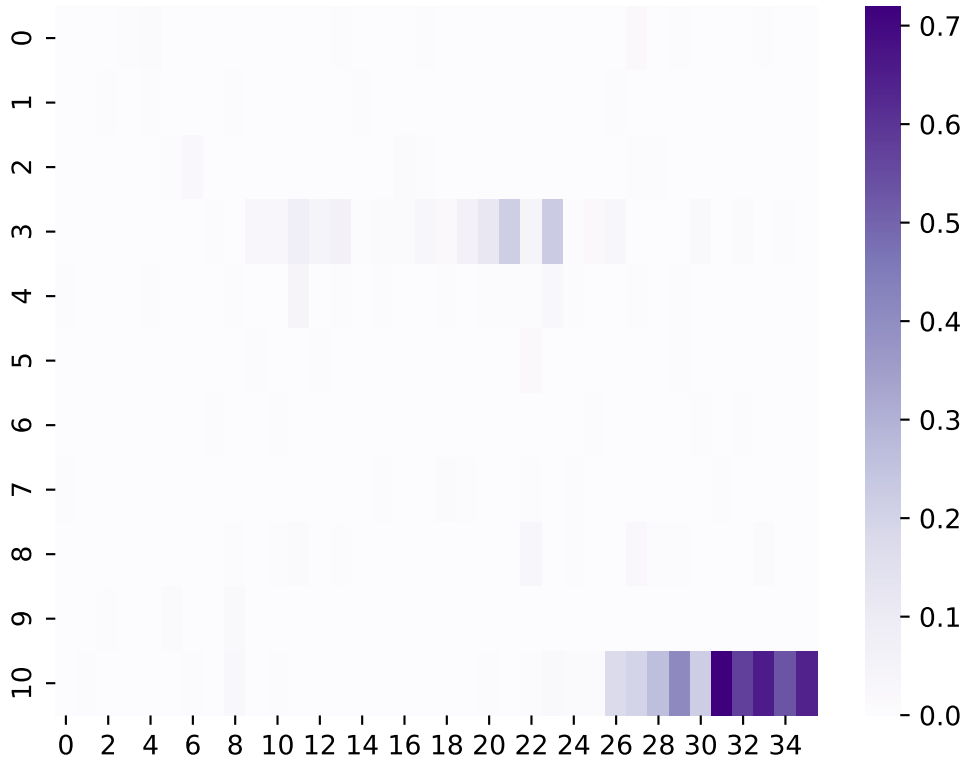


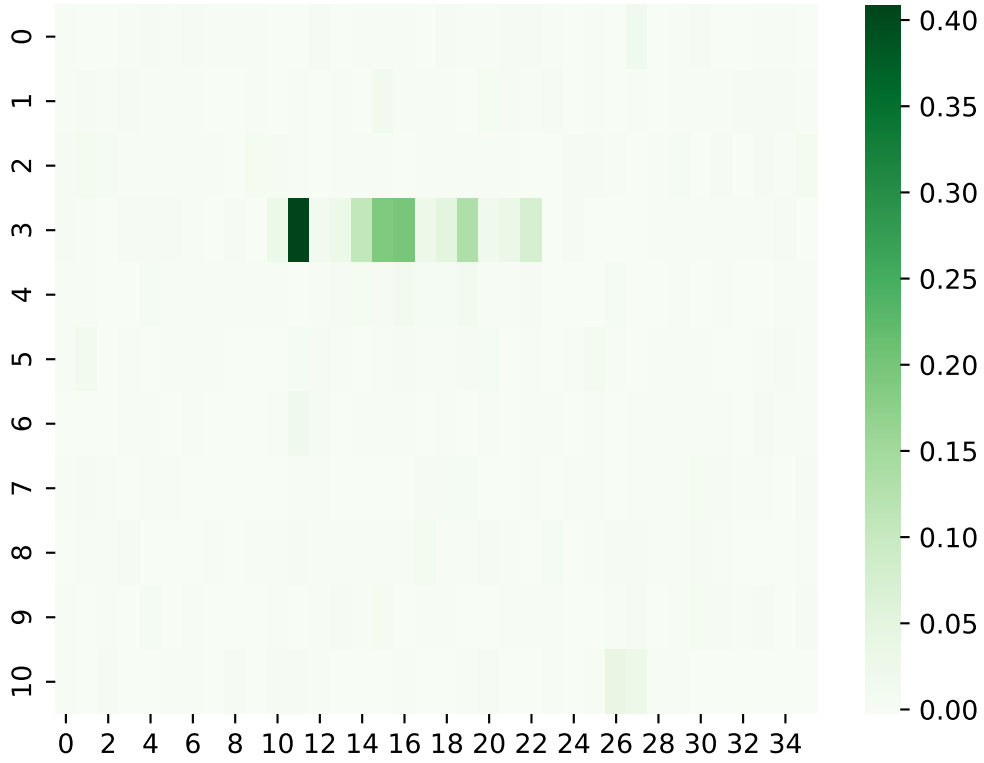

0
1
2
3
4
5
6
7
8
9
10
0
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
32
34
36
38
40
42
44
46
0.4
0.3
0.2
0.1
0.0

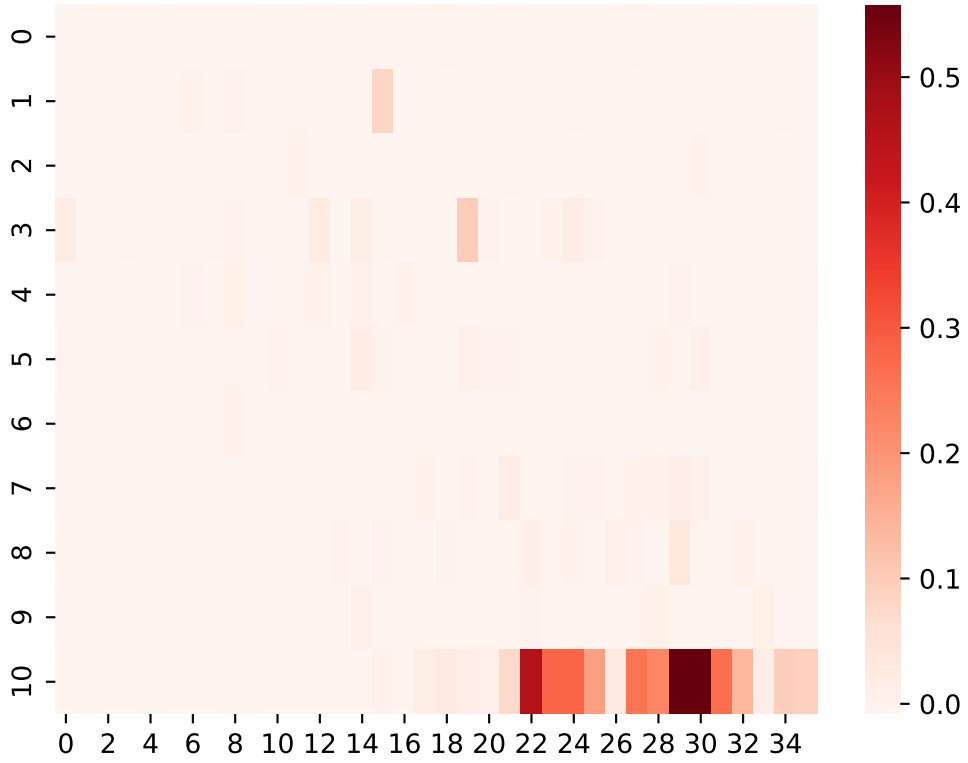

0
1
2
3
4
5
6
7
8
9
10
0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46
0.6
0.5
0.4
0.3
0.2
0.1
0.0

4.2 d

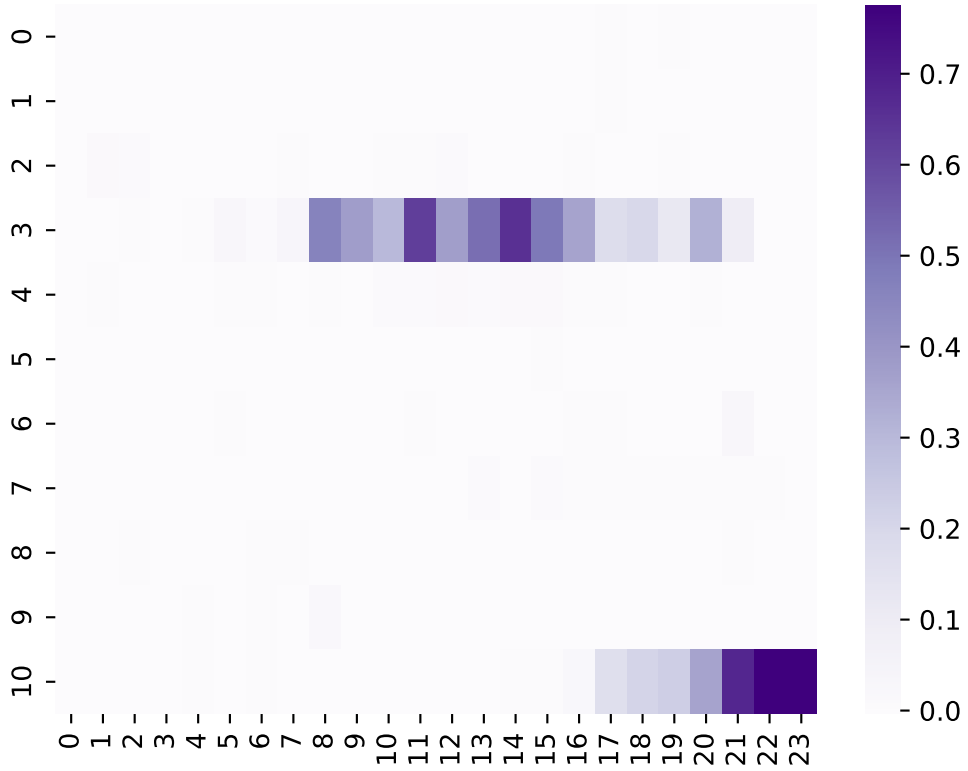
Plot for Large GPT-2 Model

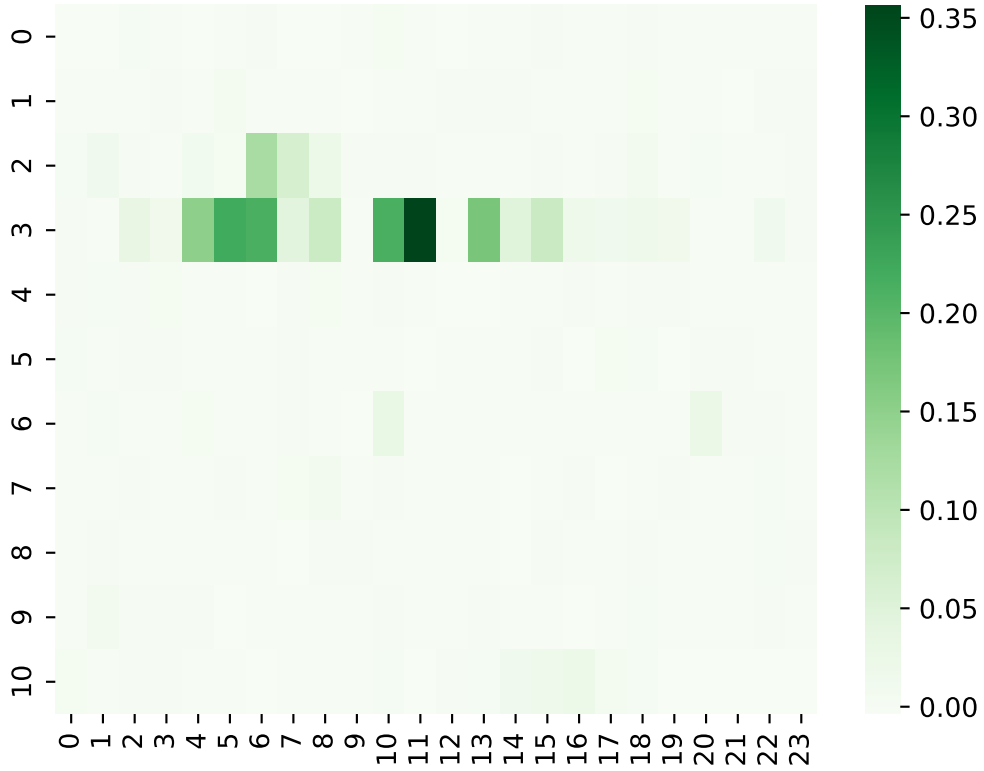


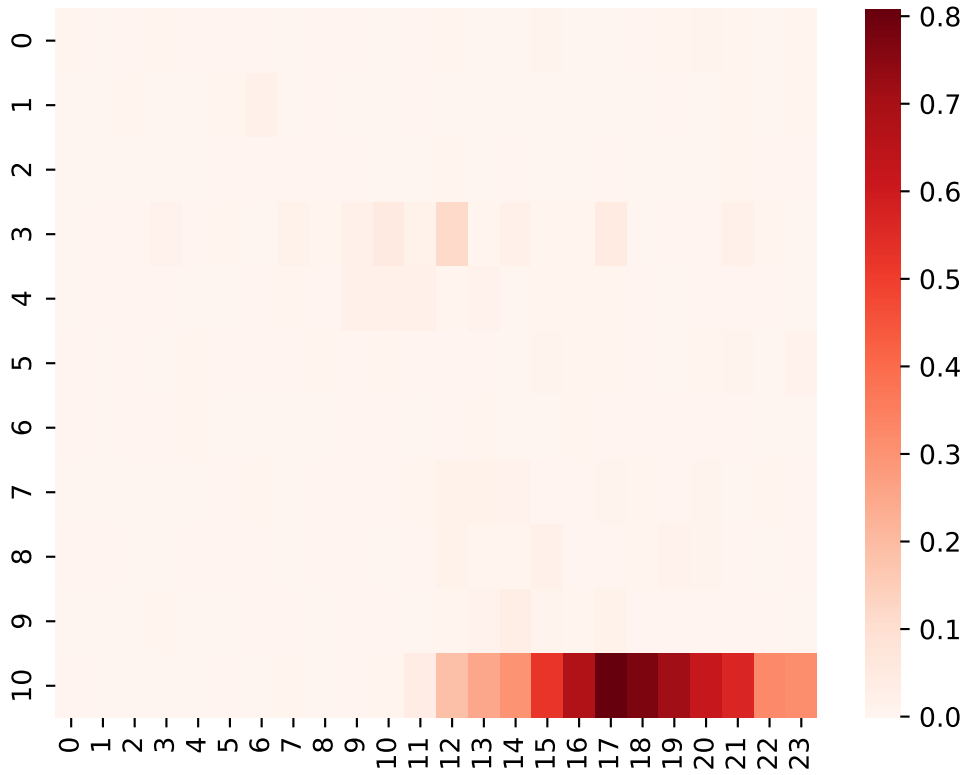

0
1
2
3
4
5
6
7
8
9
10
0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00



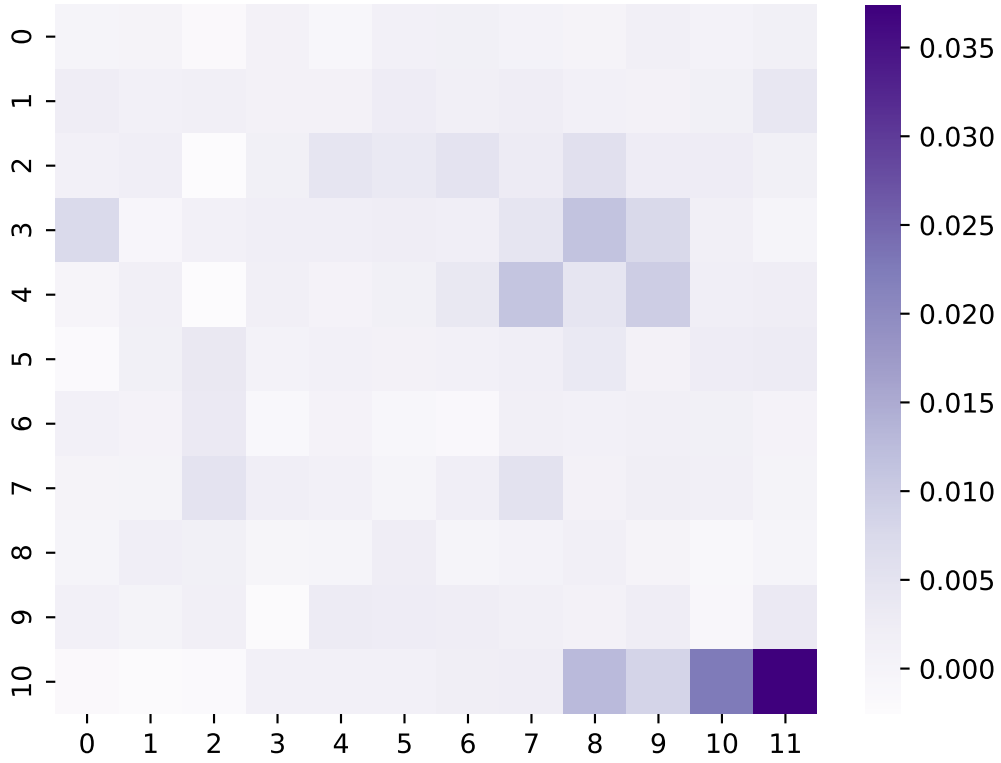
Plot for Medium GPT-2 Model

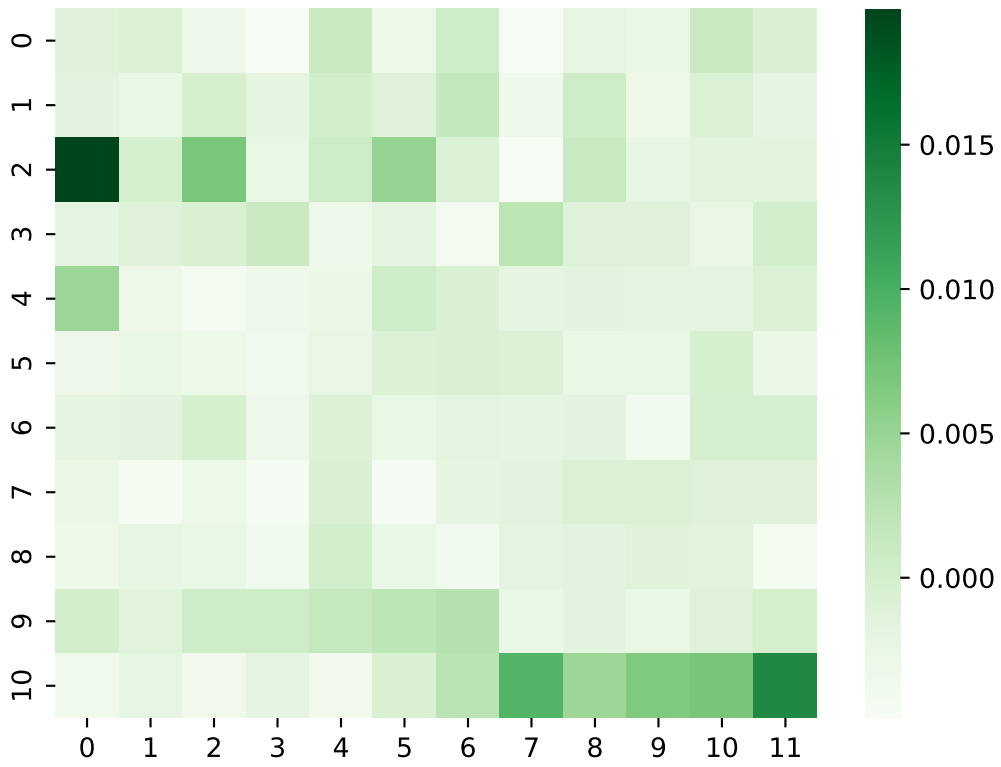

0
1
2
3
4
5
6
7
8
9
10
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0

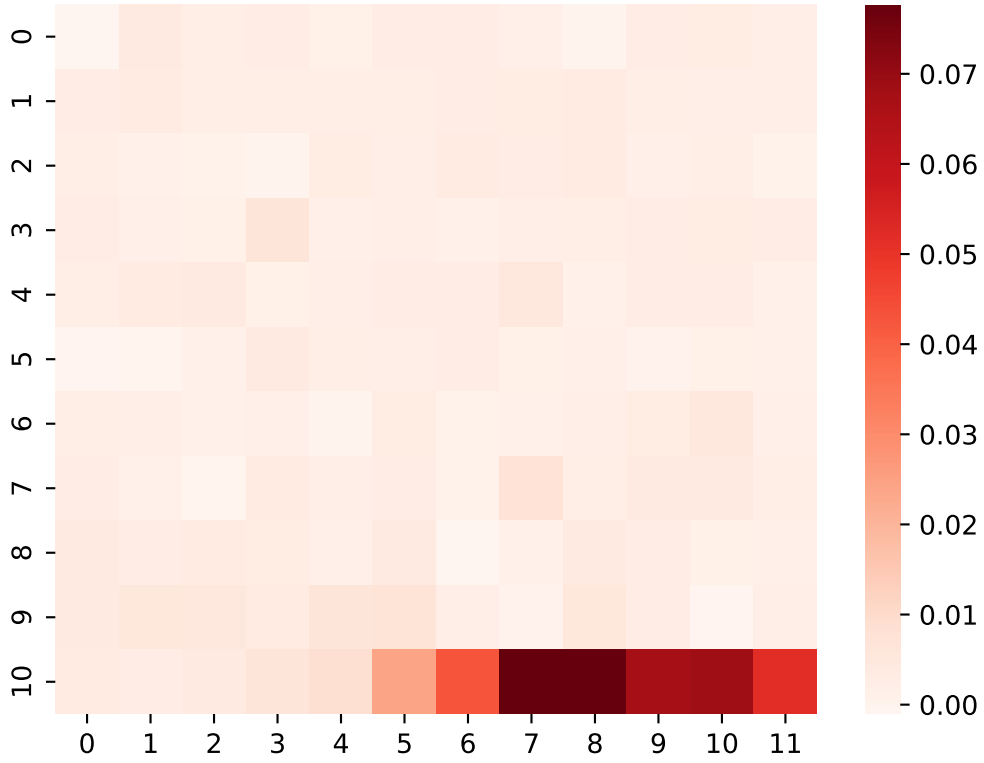

0
1
2
3
4
5
6
7
8
9
10
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00



Plot for Small GPT-2 Small


0
1
2
3
4
5
6
7
8
9
10
0
1
2
3
4
5
6
7
8
9
10
11
0.035
0.030
0.025
0.020
0.015
0.010
0.005
0.000


0
1
2
3
4
5
6
7
8
9
10
0
1
2
3
4
5
6
7
8
9
10
11
0.015
0.010
0.005
0.000

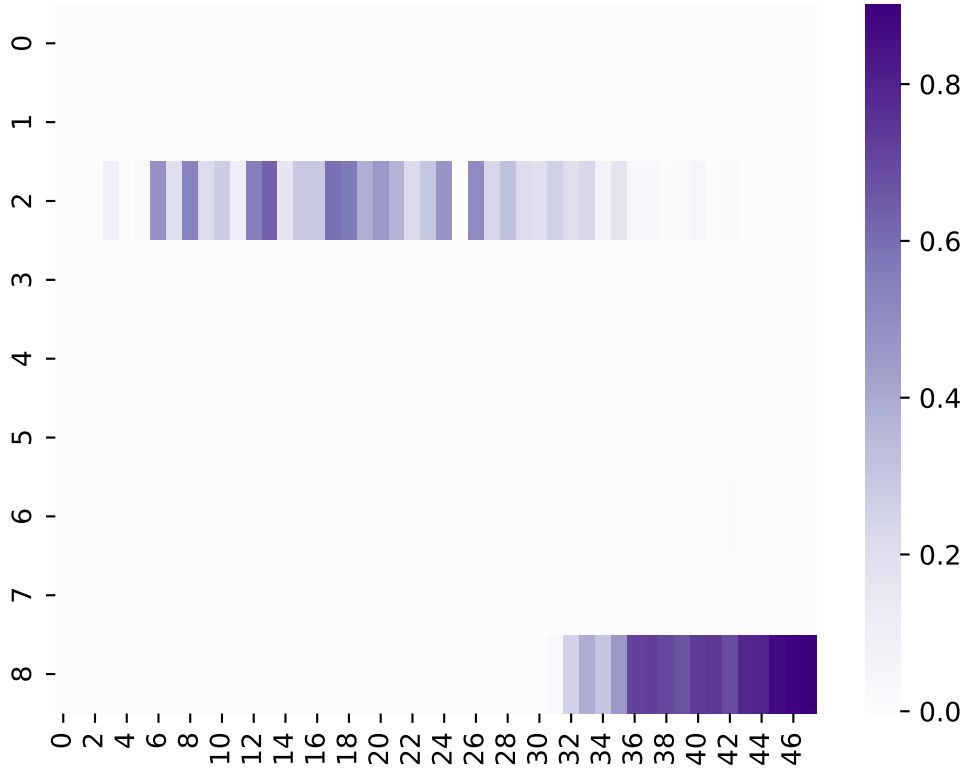


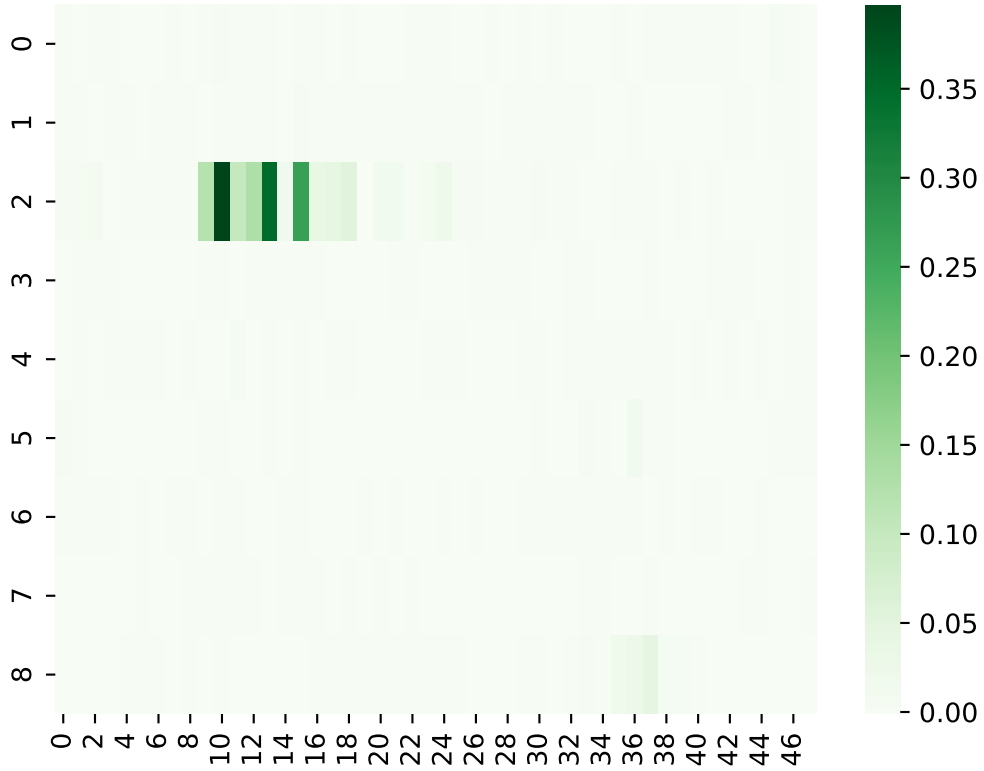
Based on the 12 different plots that I generated with the prompt "The Eiffel Tower is located in the city of" and target "Paris" among different sizes of GPT-2 model (3 from part c and the above 9), I can conclude that the causal tracing pattern no longer appears with the GPT-2 small model size for MLP where the colored squares are scattered all around the plot.
At the same time, state and attention appears to be better compared to MLP where there is a clear and distinct horizontal colored band at the bottom right corner of the plot but there are still colored squares scattered all over the plot so I have concluded that the causal tracing pattern no longer appears at the model size of small.
I think we observe this behavior because GPT-2 small has less layers so its ability to understand, form and show robust causal relationships are weaker compared to larger models of GPT-2 which has more layers to store information and relationships.
Moreover, due to the limitation in parameter space, the model is not able to create specialized neurons for storing specific information. Instead, it has to

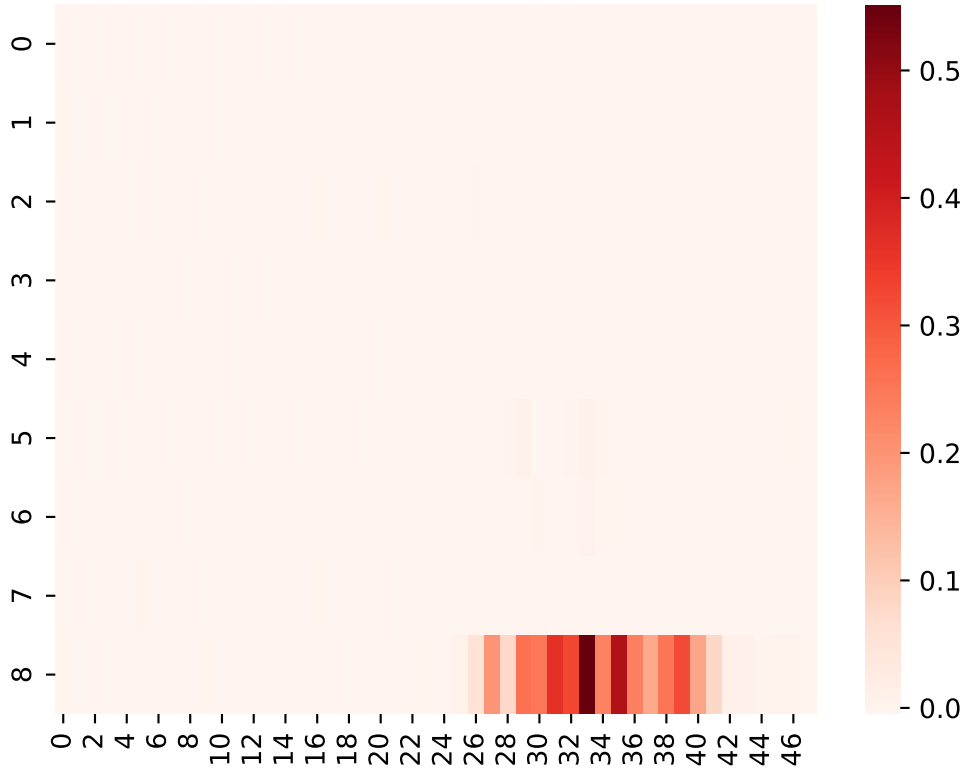
squeeze multiple pieces of information into one neuron, causing it to lose the ability to clearly distinguish between different relationships in the data. This compression of information is the cause of weaker and less clear causal tracing patterns, as the model cannot effectively track how one part of the sentence influences another.
In contrast, larger models have more layers to store and process specific information and relationships between words in the sentence which can make the causal tracing pattern more observable.

4.3 e

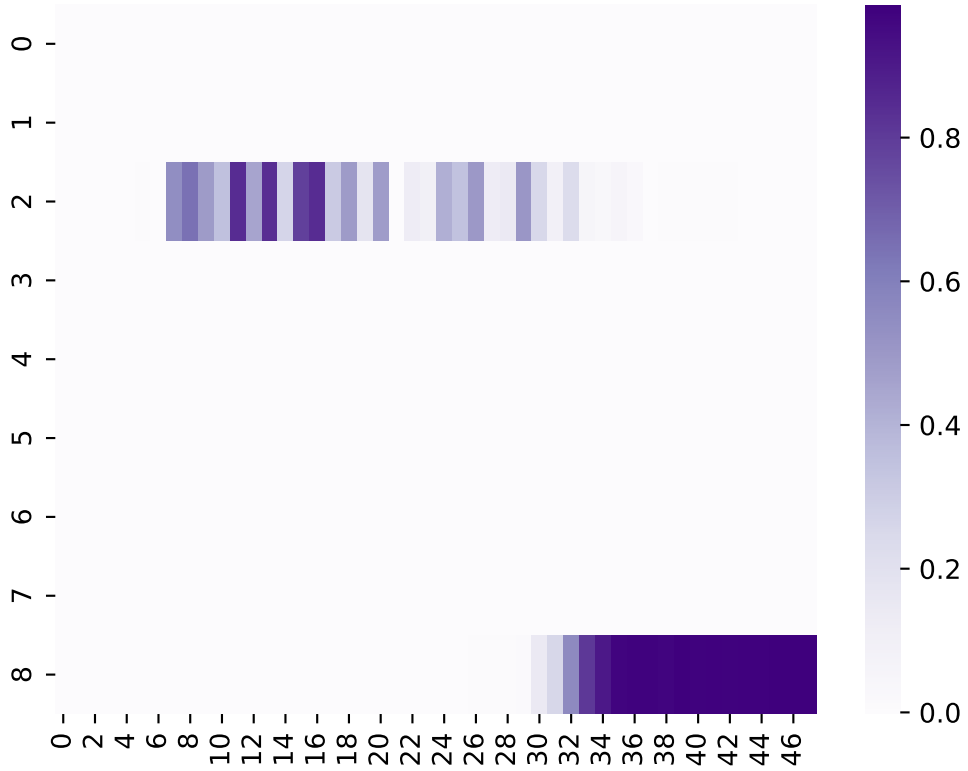
The first prompt that I use:
Prompt: The Acropolis is located in the city of
Source: The Acropolis
Target: Athens

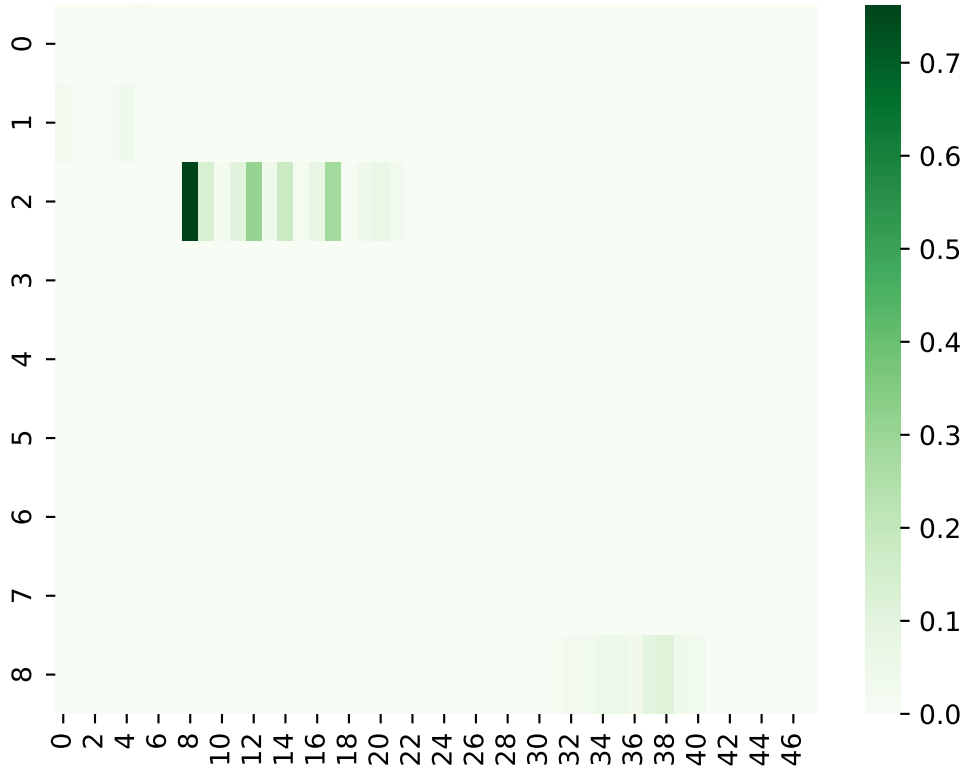


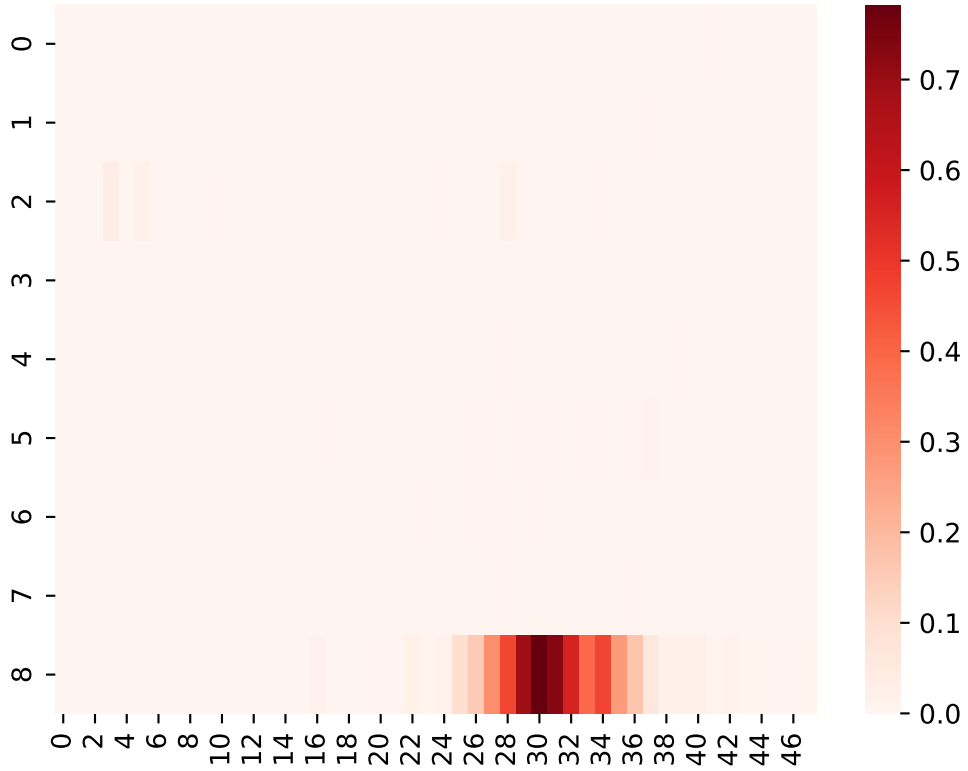

0
1
2
3
4
5
6
7
8
0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00



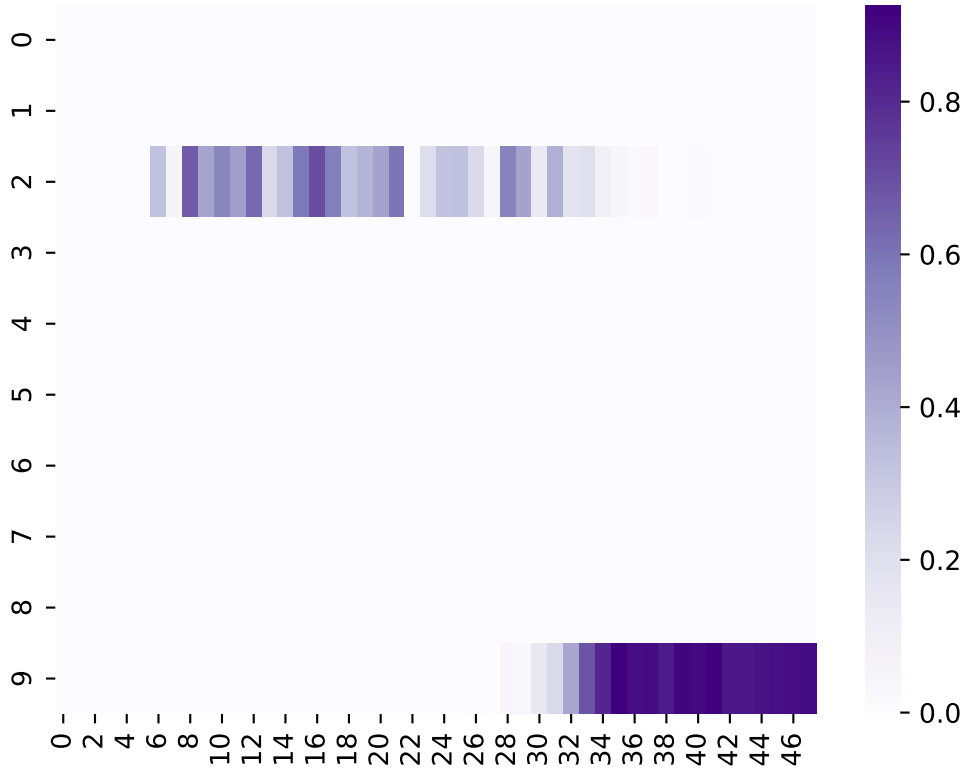
The second prompt that I use:
Prompt: The Parliament Hill is located in the city of
Source: The Parliament Hill
Target: Ottawa

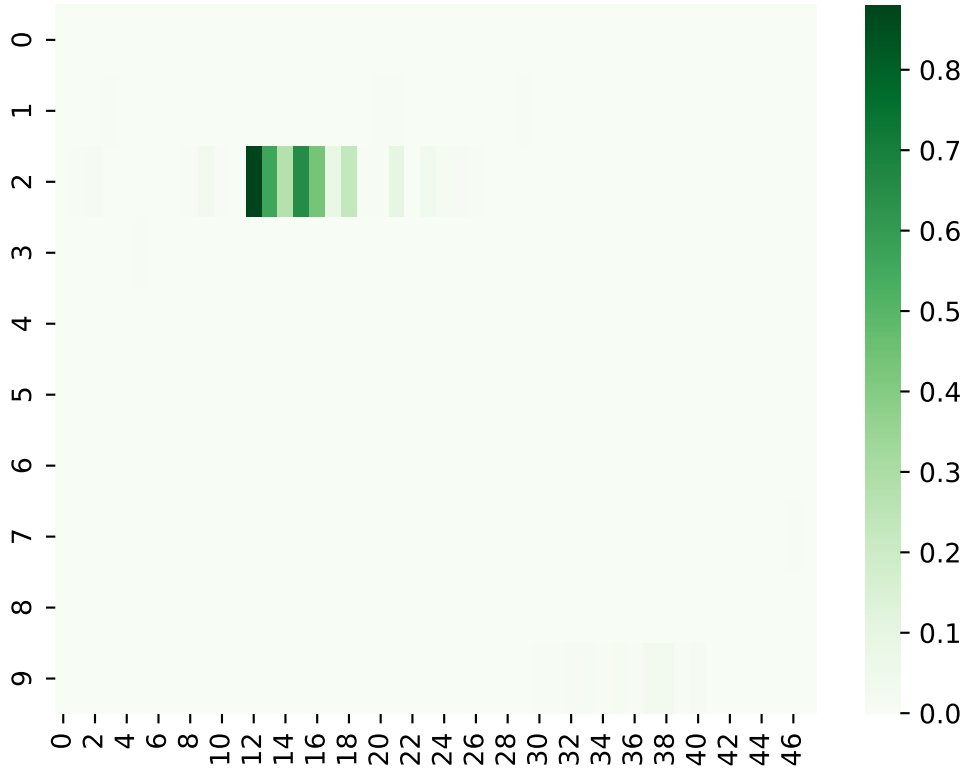

0
1
2
3
4
5
6
7
8
0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46
0.0
0.2
0.4
0.6
0.8

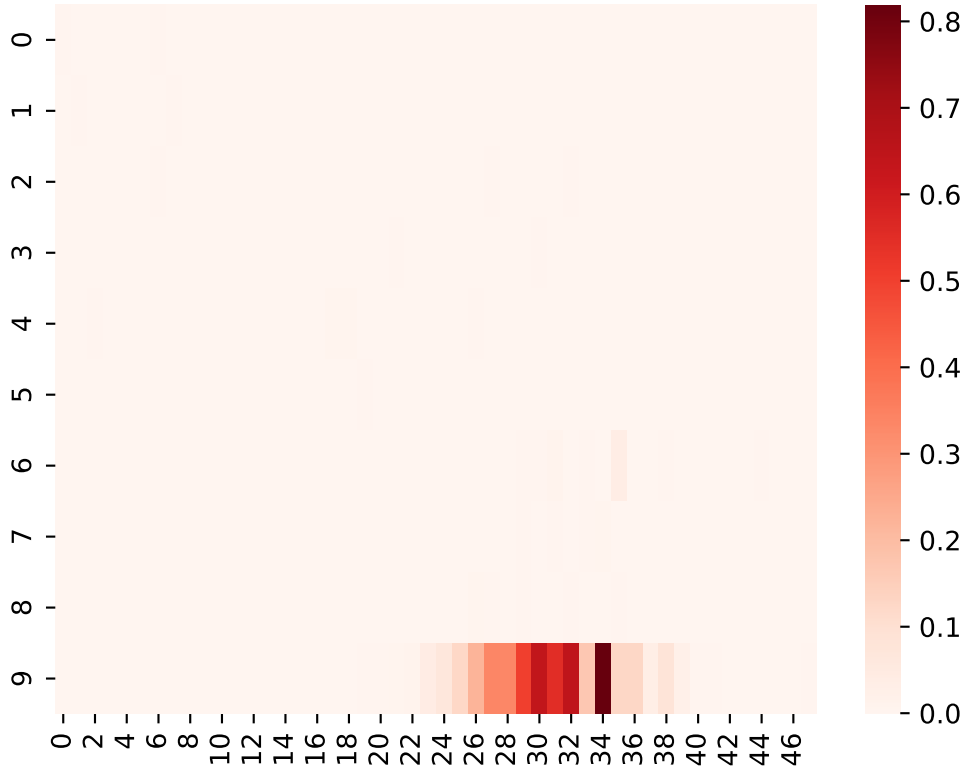

0
1
2
3
4
5
6
7
8
0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0



The third prompt that I use:
Prompt: The Rideau Canal is located in the city of
Source: The Rideau Canal
Target: Ottawa


0
1
2
3
4
5
6
7
8
9
0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46
0.0
0.2
0.4
0.6
0.8


0
1
2
3
4
5
6
7
8
9
0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0

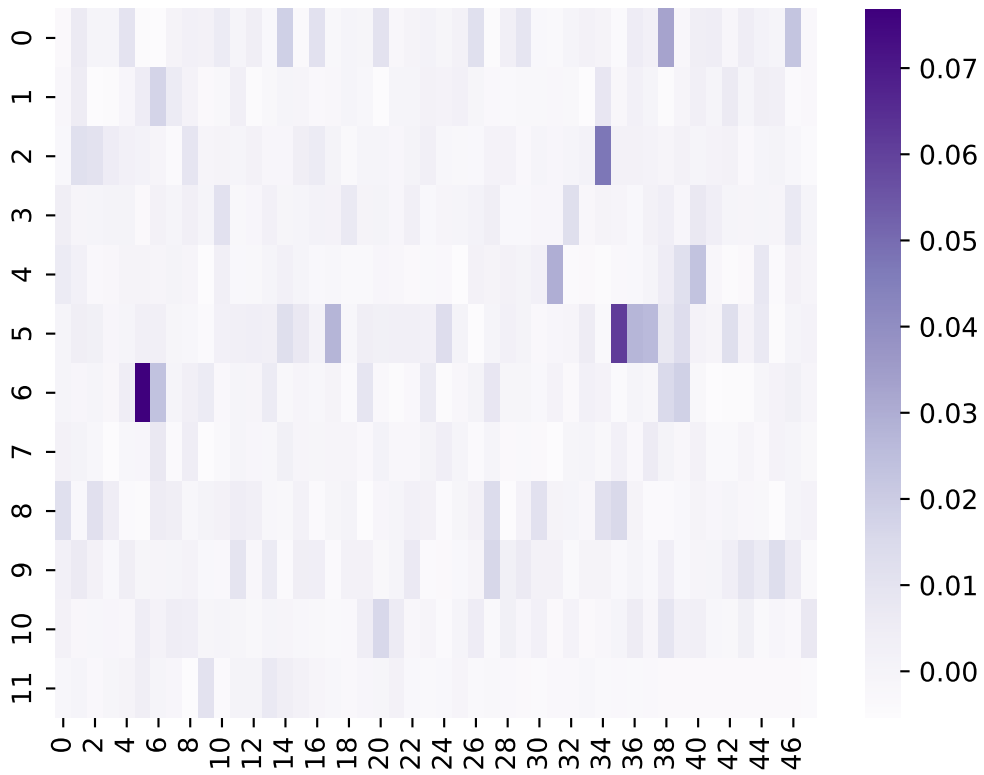


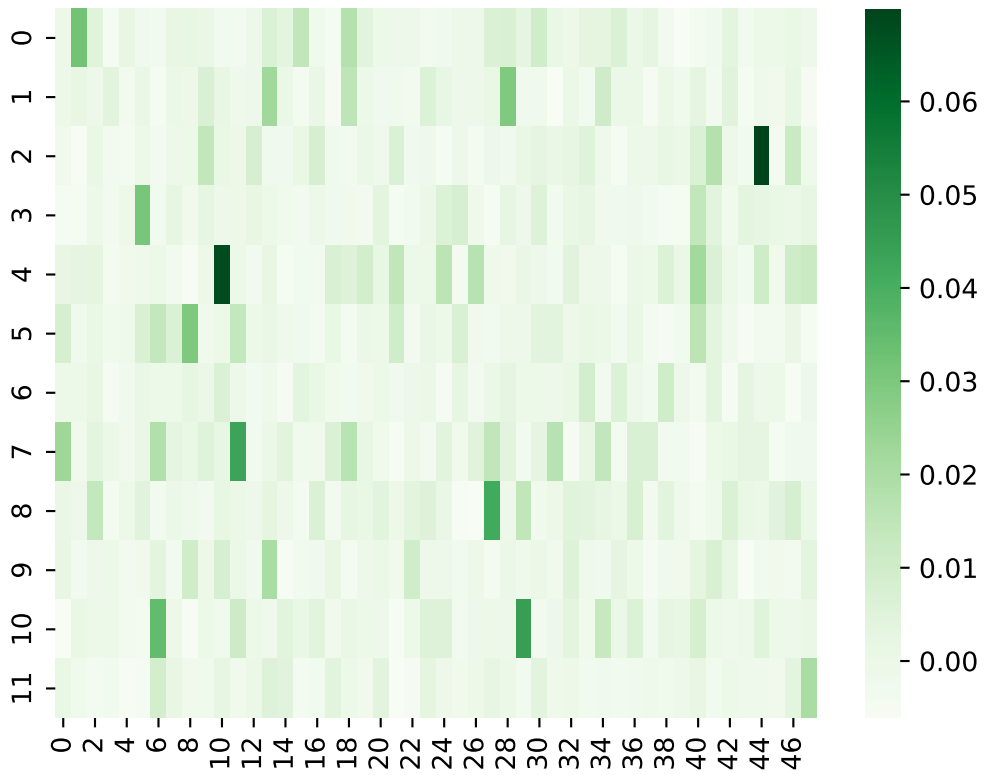
I have tried running the causal tracing analysis with the above 3 prompts that I found and I got plots that follows similar pattern as Figure 2.
I have included both the example that I use as well as the plot produced.
I found that these sentences that I observed a clear causal tracing pattern shares some common characteristics and I believe these characteristics contribute to the clear causal tracing pattern.
The first characteristic is that all these sentences have the same grammatical structure compared to the examples given to us (The Eiffel Tower and the Space Needle example) where we only replace the landmark city pair.
For example, the first prompt uses Acropolis instead of Eiffel Tower and use the city that Acropolis locates in instead of Paris.
By following the same structure, the model will get the same contextual cues which allows us to observe a causal tracing pattern similar to Figure 2 across different example prompts that we came up.
The second and possibly the most obvious characteristic is the fact that the relationship between the landmark and the city must be correct.

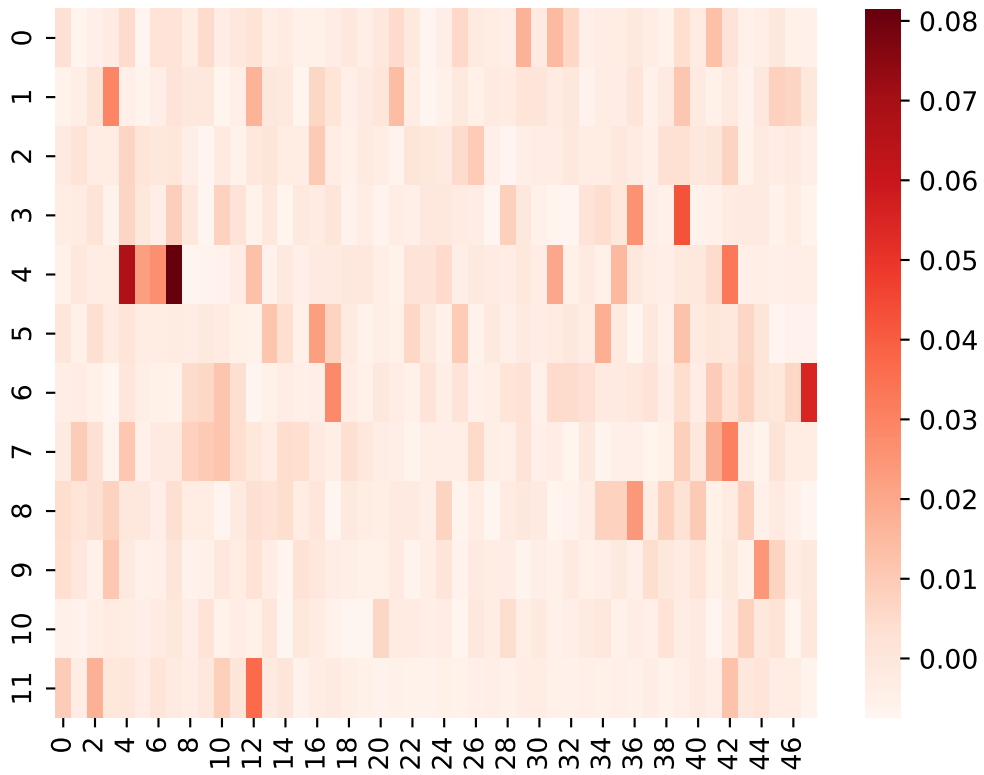
For example, you can't say Eiffel Tower is located in the city of Toronto.
If we give an incorrect landmark and city pair, then there is really no clear cause and effect connection within the sentence so the model will not reliably trace the relationship between the landmark and city which consequently end up with no clear causal tracing pattern.
The third and last characteristic that I observe is the landmark should be globally recognizable and unique and the city should be strongly associated or linked to the landmark.
For example, Eiffel Tower is globally recognizable and unique and whenever people think of the location of Eiffel Tower, they will immediately identify that it is located in the city of Paris.
Another example will be the Space Needle example where it is globally recognizable and unique and people will immediately identify it is located in city of Seattle whenever they think of the location of Space Needle.
By following this characteristic, we can make sure that there is no unambiguous association given the landmark name (Ex. given the name of the landmark, there is only one city that could be the answer). This ensures that there is a clear and unique relationship between the landmark and city so we can observe a clear causal tracing pattern.
To sum up my observation, to observe a similar causal tracing patterns compared to Figure 2, we need the structure of the sentence to be similar by only replacing the landmark and city pair.
In addition to sentence structure, we also need to ensure that the name of the landmark is well-known and unique and the relationship between landmark and city is globally recognizable and unique (one-to-one relationship).
Lastly, we need to make sure the landmark and city pair is correctly associated (not associate the landmark to the wrong city and vice versa).

4.4 f

The first Prompt that I came up with is a counterexample to the third characteristic that I mention in part e where I still use the same sentence structure but this time use a landmark and city pair that is not unique and globally recognized.
Prompt: The Church of St. Mary is located in the city of
Source: The Church of St. Mary
Target: London

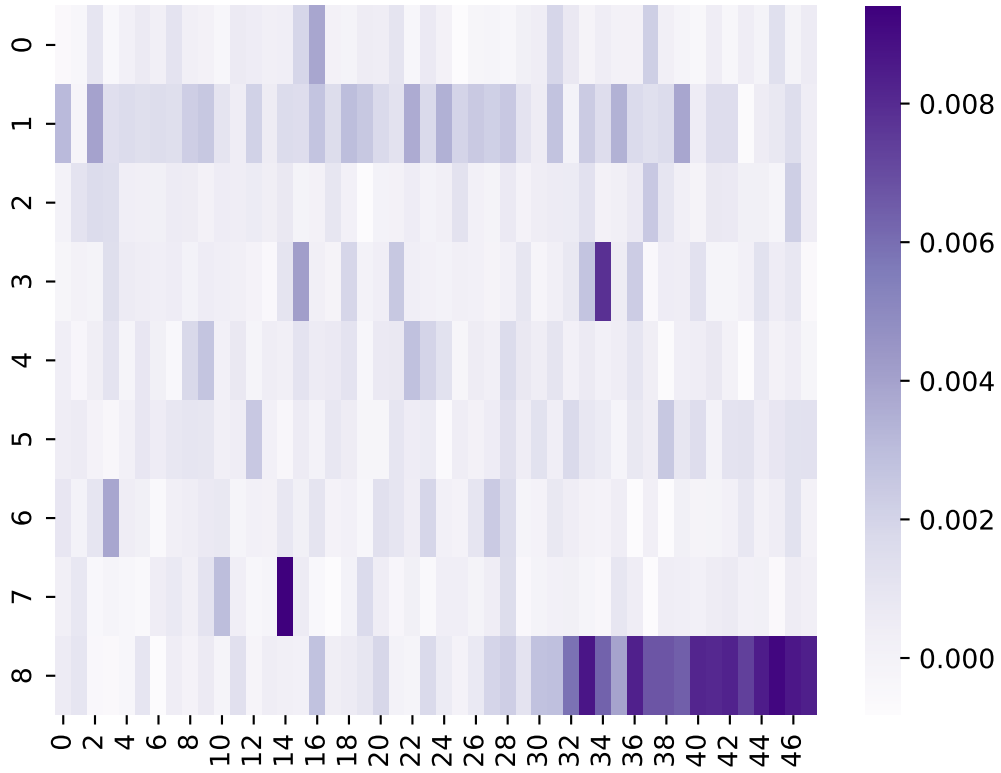

0
1
2
3
4
5
6
7
8
9
10
11
0
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
32
34
36
38
40
42
44
46
0.07
0.06
0.05
0.04
0.03
0.02
0.01
0.00

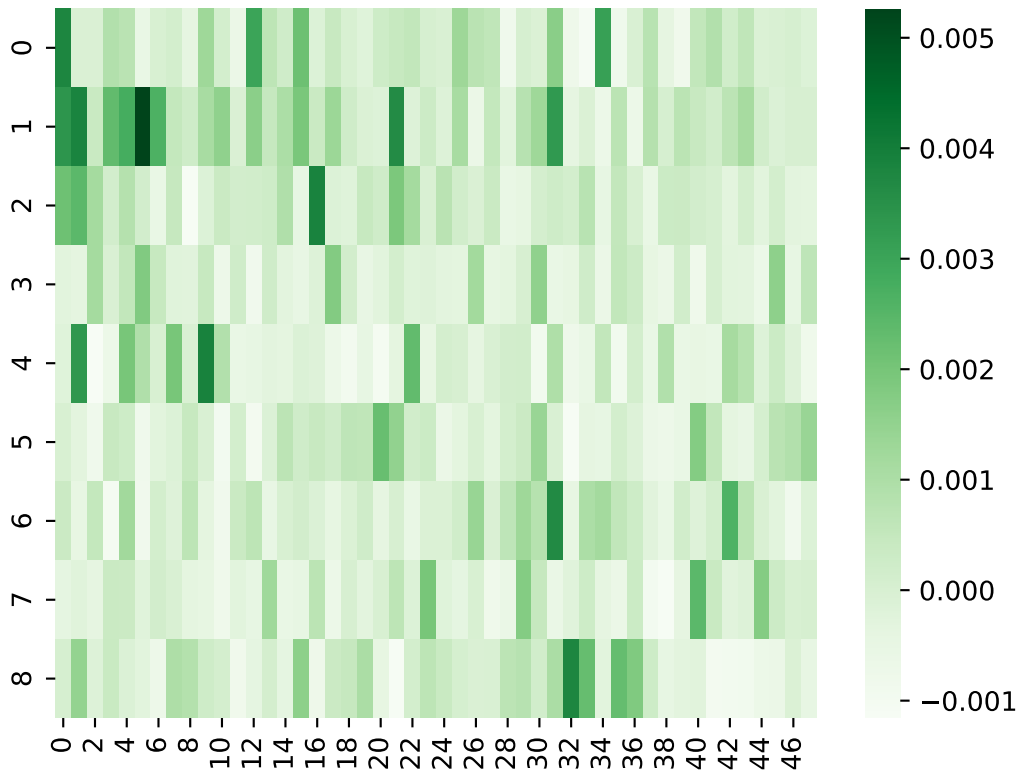

0
1
2
3
4
5
6
7
8
9
10
11
0
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
32
34
36
38
40
42
44
46
0.06
0.05
0.04
0.03
0.02
0.01
0.00

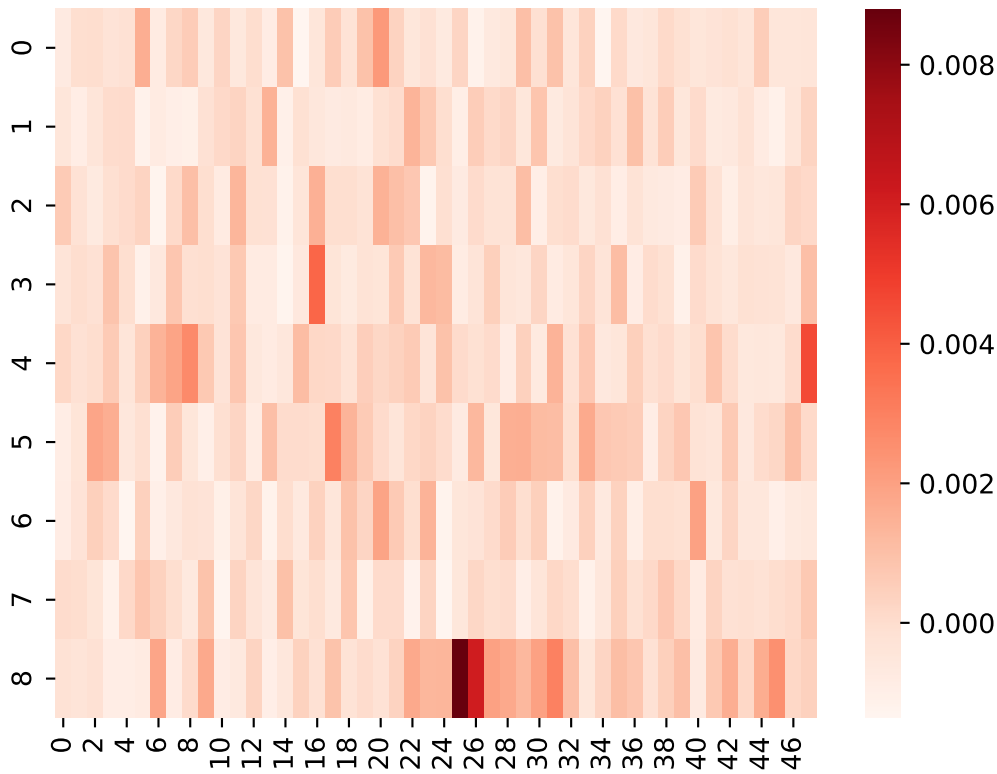


We can see that the plot does not form any clear and colored horizontal band and all colored rectangle are scattered all around the plot so the causal tracing pattern does not exist for this prompt.
In this example prompt, the landmark Church of St. Mary is not globally recognizable and it is not unique since there are multiple churches with the same name across different cities so the model is confused on which church it refers to (many-to-many relationship).
In addition, there is no clear association between the church name and the city of London so there is no link between the landmark and the city.
With these analysis, we can see that the sequence that we corrupt does not directly influence or leads to the prediction of London so we cannot observe a clear causal tracing pattern with this example prompt.
The second Prompt that I cam up with is a similar example to the first one where there is multiple possible predictions given the source so the model can get some information on the target word based on the source but there is not a direct influence on the target word from the source.

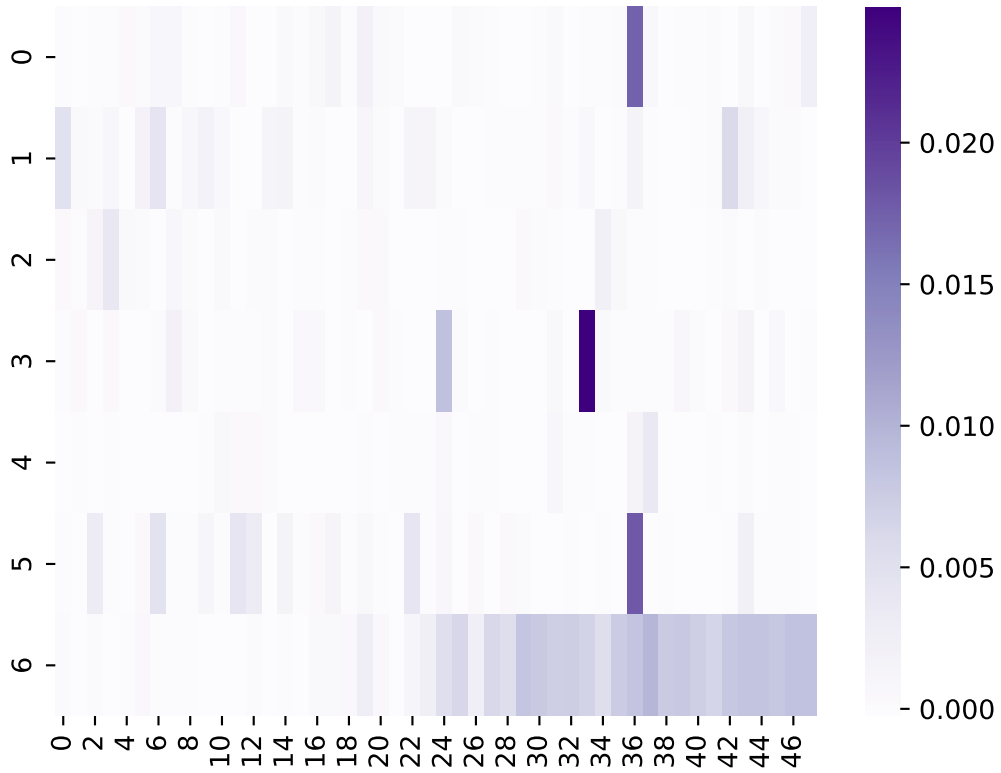
Prompt: The Olympic game is hosted in the city of
Source: The Olympic game
Target: Tokyo

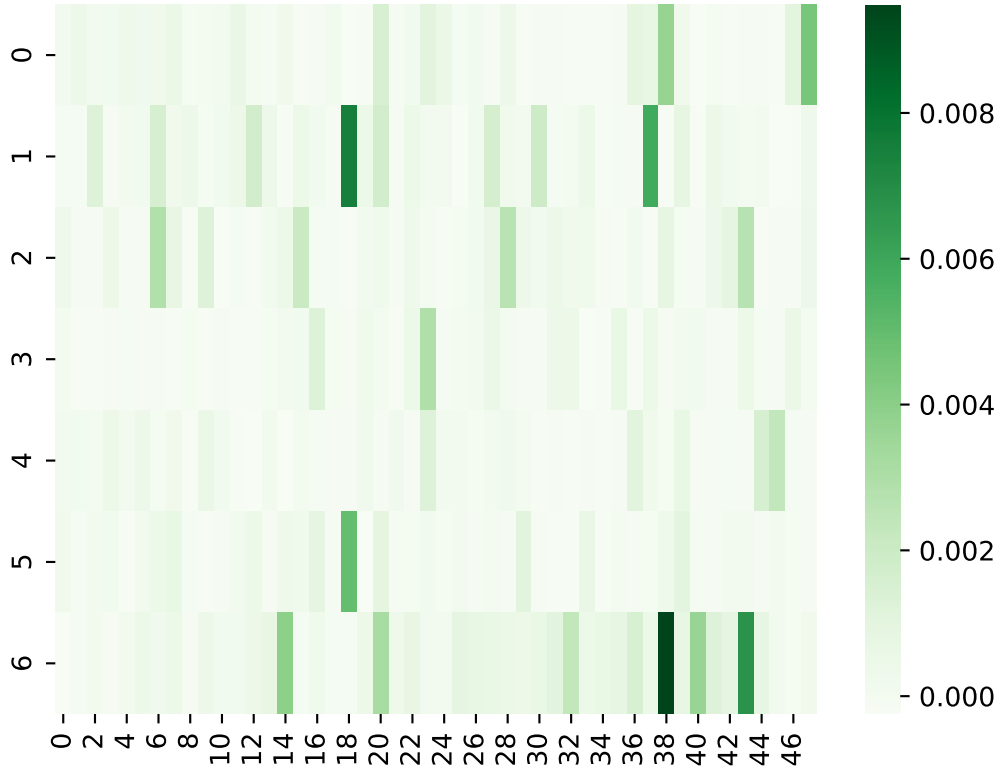


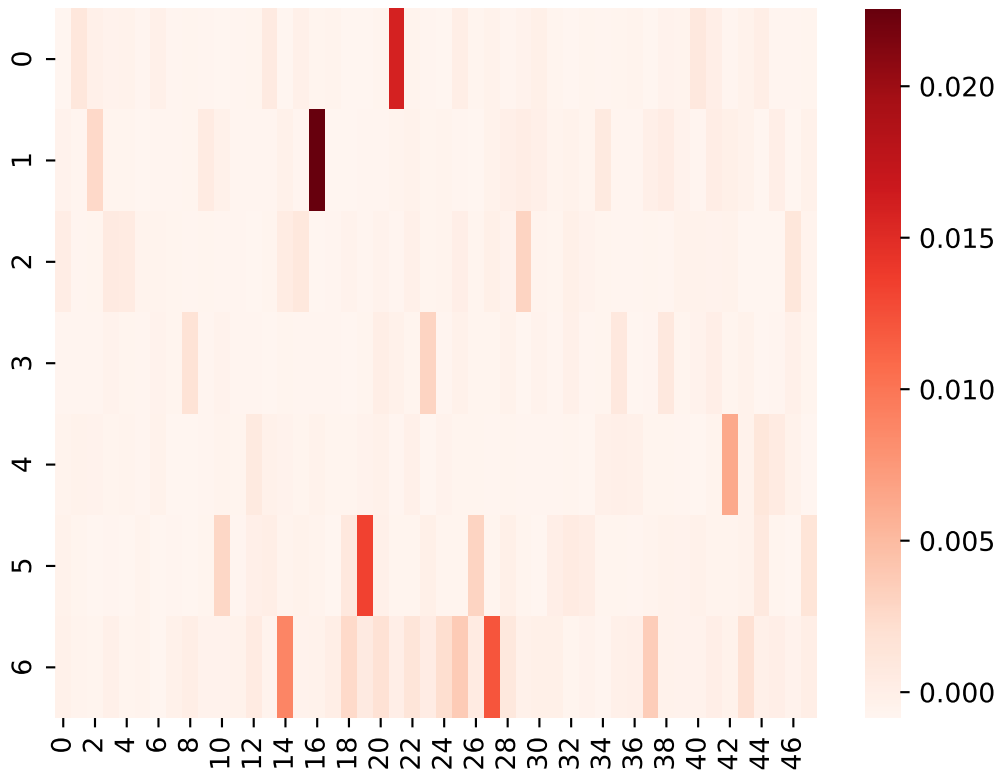

0
1
2
3
4
5
6
7
8
0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46
0.005
0.004
0.003
0.002
0.001
0.000
−0.001



We can see that there is still no causal tracing pattern in the plot where all colored rectangles are scattered all around the plot.
I believe this is because the model can not make a prediction directly from the source which is Olympic game.
The city that host the Olympic game varies so there is not a fixed city that host all the Olympic games so without further context such as which year of Olympic, the model is not able to make a confident enough prediction which leads to no clear causal tracing pattern.
The third Prompt that I came up with is an example where there is completely no relationship between the source and the target word or in other words, we cannot make any prediction on the target word from the source.
Prompt: The weather was cold and he watched
Source: The weather was cold
Target: television


0
1
2
3
4
5
6
0
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
32
34
36
38
40
42
44
46
0.020
0.015
0.010
0.005
0.000


0
1
2
3
4
5
6
0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46
0.008
0.006
0.004
0.002
0.000



We can see from the heatmap plot that there is no clear causal tracing pattern for this prompt.
In this prompt, there is no relationship between the two part of the sentence and there is no relationship between the source and the target word that we are trying to predict.
I believe this is the cause of the absence of causal tracing pattern where if there does not exist a relationship between the corrupted sequence and the target word, then of course there will be no causal tracing pattern observed.
The plot produced by these three prompts combined with the three prompts from part e gives me some insights on how language models process, store and generate factual information obtained from pretraining.
I realized that language models rely on statistical relationship between words that they observed from pretraining to make predictions.
For example, the model rely on the fact that Eiffel Tower is strongly associated with the city Paris to make a strong predictions on the Eiffel Tower example.
However, when it comes to the examples in part f, the model seems to be con-

fused on their prediction since there is no clear relationship that can directly lead to the correct target word.
So language model rely on the pretraining dataset to give the model sufficient statistical relationship between words to make a correct prediction.
This means that language model will process the dataset used in pretraining into list of statistical relationships between words and then store them within the model to make prediction.
Whenever, the model observes a word, it will examine all the relationships stored in the model and assigns a probability to each possible target word that represents how confident the prediction is.
As a result, if the sentence has a strong relationship between the source and the target word, then the model will make a strong prediction and vice versa.
Overall, the performance of language models not only rely on the algorithm used for the model but also on the dataset used in pretraining as well as the sentence that is given to the model (whether there is a strong relationship between the source adn target where the model can directly make a prediction of target word from source).